



Язык программирования C++

Лекция 5

ОБРАБОТКА ОШИБОК

обработать ошибку в функции?

передать ошибку
вызывающему?

остановить программу?

выбросить исключение?

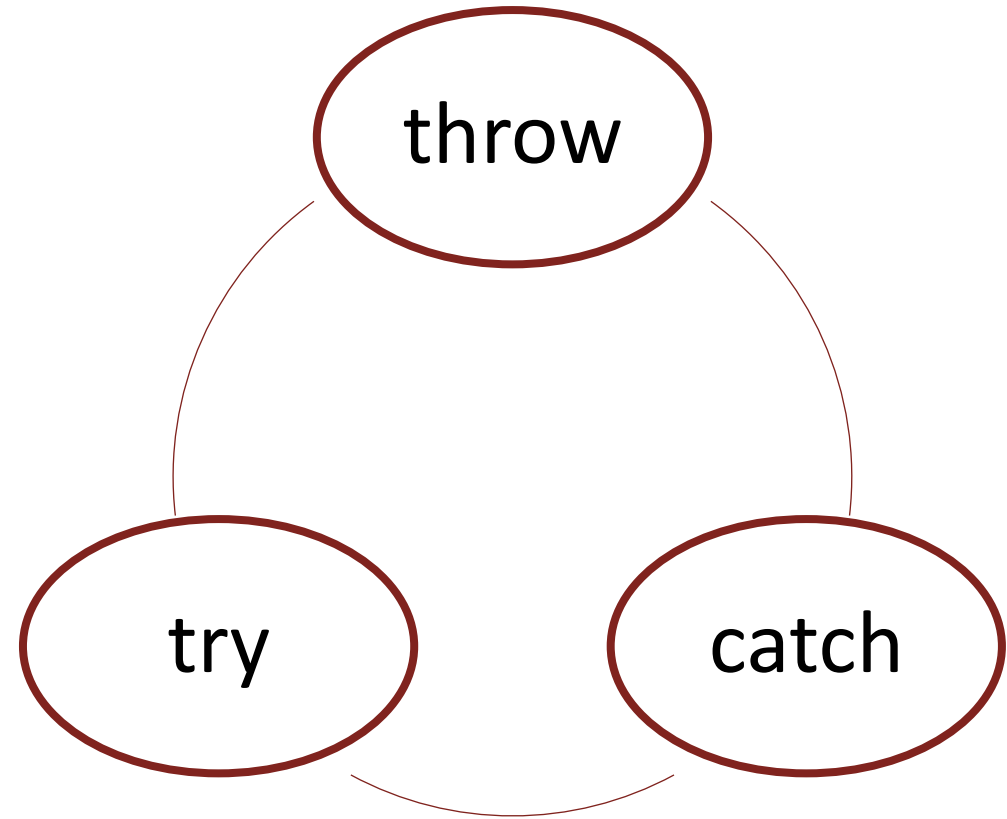
```
✓ void divide(int x, int y)
{
    if (y != 0)
        cout << x / y;
    else
        cout << "Error";
}
```

СТРУКТУРИРОВАННАЯ ОБРАБОТКА ИСКЛЮЧЕНИЙ

Дефекты

Пользовательские
ошибки

Исключительные
ситуации



THROW

```
throw - 1;
```

```
throw ENUM_INVALID_INDEX;
```

```
throw "Can not take square root of negative number";
```

```
throw MyException("Fatal Error");
```

TRY-CATCH

```
try
{
    //код, который может сгенерировать исключение
}
catch
{
    //код, обрабатывающий исключение
}
```

```
try
{
    //
    throw -1;
}
catch (int x)
{
    std::cerr << "int exception: " << x << '\n';
}
catch (double)
{
    std::cerr << "double exception" << '\n';
}
```

```
double x{};
cin >> x;
try
{
    if (x < 0.0)
    {
        throw "Can not take sqrt of negative number";
    }
    std::cout << "The sqrt of " << x << " is " << std::sqrt(x) << '\n';
}
catch (const char* exception)
{
    std::cerr << "Error: " << exception << '\n';
}
```

```
✓ double mySqrt(double x)
{
    if (x < 0.0)
        throw "Can not take sqrt of negative number";

    return std::sqrt(x);
}
```

```
try
{
    double d = mySqrt(x);
    std::cout << "The sqrt of " << x << " is " << d << '\n';
}
catch (const char* exception)
{
    std::cerr << "Error: " << exception << std::endl;
}
```

УНИВЕРСАЛЬНЫЙ ОБРАБОТЧИК

```
try
{
    throw 5;
}
catch (double x)
{
    std::cout << "We caught an exception of type double: " << x << '\n';
}
catch (...)
{
    std::cout << "We caught an exception of an undetermined type\n";
}
```



```
int& IntArray::operator[] (const int index)
{
    if (index < 0 || index >= getLength())
        throw index;

    return m_data[index];
}
```

```

class Member
{
public:
    Member()
    {
        std::cerr << "Member allocated some resources\n";
    }

    ~Member()
    {
        std::cerr << "Member cleaned up\n";
    }
};

```

```

try
{
    A a{ 0 };
}
catch (int)
{
    std::cerr << "Oops\n";
}

```

```

class A
{
private:
    int m_x{};
    Member m_member;

public:
    A(int x) : m_x{ x }
    {
        if (x <= 0)
            throw 1;
    }

    ~A()
    {
        std::cerr << "~A\n";
    }
};

```

Member allocated some resources
Member cleaned up
Oops

```
class ArrayException
{
private:
    std::string m_error;

public:
    ArrayException(std::string error)
        : m_error{ error }
    {
    }

    std::string_view getError() const { return m_error; }
};
```

```
try
{
    int value{ array[5] };
}
catch (const ArrayException& exception)
{
    std::cerr << "An array exception occurred (" << exception.getError() << ")\n";
}
```

STD::EXCEPTION

```
try
{
    std::string s;
    s.resize(std::numeric_limits<std::size_t>::max());
}
catch (const std::exception& exception)
{
    std::cerr << "Standard exception: " << exception.what() << '\n';
}
```

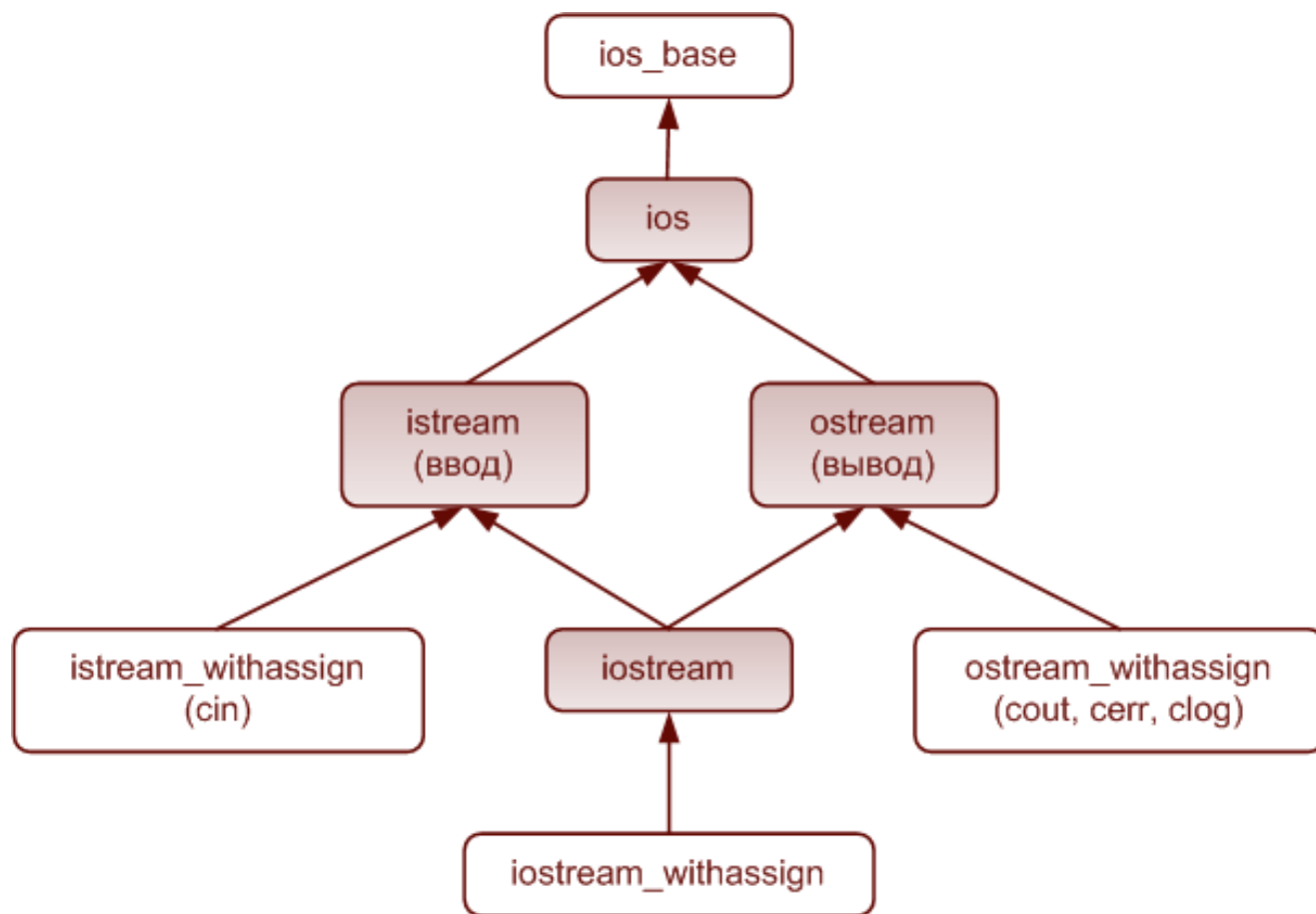
Standard exception: string too long

СОБСТВЕННЫЙ КЛАСС ИСКЛЮЧЕНИЙ

```
✓class ArrayException : public std::exception
{
private:
    std::string m_error{};

public:
    ✓ArrayException(std::string_view error)
        : m_error{ error }
    {
    }
    const char* what() const noexcept override { return m_error.c_str(); }
};
```

ПОТОКИ ВВОДА/ВЫВОДА



cin – класс, привязанный к стандартному вводу

cout – класс, связанный со стандартным выводом

cerr – класс, связанный со стандартным выводом сообщений об ошибках, обеспечивающий небуферизованный вывод

clog – класс, связанный со стандартным выводом сообщений об ошибках, обеспечивающий буферизованный вывод

ISTREAM

```
char input[10];  
std::cin >> input;
```

```
std::cin >> std::setw(10) >> input;
```

```
char ch;  
while (std::cin >> ch)  
    std::cout << ch;
```

```
while (std::cin.get(ch))
```

```
char input[10];  
std::cin.getline(input, 10);  
std::cout << input << '\n';  
std::cin.get(input, 10);  
std::cout << input << '\n';
```

ignore()

ignore(int nCount)

peek()

unget()

putback(char ch)

OSTREAM

```
std::cout.setf(std::ios::showpos);  
std::cout << 22;
```

```
std::cout.unsetf
```

```
(std::ios::showpos | std::ios::uppercase)
```

```
std::cout.unsetf(std::ios::dec);  
std::cout.setf(std::ios::hex);
```

```
std::cout.setf(std::ios::hex, std::ios::basefield);
```

```
std::boolalpha
```

```
std::fixed
```

```
std::setprecision(int)
```

```
std::setfill(char)
```

```
std::setw(int)
```

```
std::left
```

```
std::right
```

```
str()
```

```
clear()
```


СОСТОЯНИЕ ПОТОКА

goodbit

badbit

eofbit

failbit

good()	true, если установлен goodbit
bad()	true, если установлен badbit
eof()	true, если установлен eofbit
fail()	true, если установлен failbit
clear()	Очищает все флаги и восстанавливает поток в состояние goodbit
clear(state)	Очищает все флаги и устанавливает флаг состояния, переданный в параметре
rdstate()	Возвращает текущие установленные флаги
setstate(state)	Устанавливает флаг состояния, переданный в параметре

ПРОВЕРКА КОРРЕКТНОСТИ ВХОДНЫХ ДАННЫХ

<code>isalnum(int)</code>	ненулевое значение, если параметр представляет собой букву или цифру
<code>isalpha(int)</code>	ненулевое значение, если параметр представляет собой букву
<code>iscntrl(int)</code>	ненулевое значение, если параметр является управляющим символом
<code>isdigit(int)</code>	ненулевое значение, если параметр является цифрой
<code>isgraph(int)</code>	ненулевое значение, если параметр является печатным символом, который не является пробелом
<code>isprint(int)</code>	ненулевое значение, если параметр является печатным символом (включая пробелы)
<code>ispunct(int)</code>	ненулевое значение, если параметр не является ни буквенно-цифровым, ни пробельным символом
<code>isspace(int)</code>	ненулевое значение, если параметр – пробельный символ
<code>isxdigit(int)</code>	ненулевое значение, если параметр является шестнадцатеричной цифрой (0-9, a-f, A-F)

```

int main()
{
    int num{};

    while (true)
    {
        cout << "Number: ";
        cin >> num;
        if (cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        break;
    }
    cout << "Your number: " << num << '\n';
}

```

Number: qwerty
 Number: q2
 Number: 2
 Your number: 2

```

cin.ignore(numeric_limits<streamsize>::max(), '\n');
break;

```

Number: 11qwe
 Your number: 11

```
int main()
{
    int num{};

    while (true)
    {
        cout << "Number: ";
        cin >> num;
        if (cin.fail())
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            continue;
        }
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        if (std::cin.gcount() > 1)
        {
            continue;
        }
        break;
    }
    cout << "Your number: " << num << '\n';
}
```

```
#include <limits>
```

```
Number: 11qwe
Number: 1qwe
Number: 1q
Number: 11
Your number: 11
```

ФАЙЛОВЫЙ ВВОД/ВЫВОД

```
std::ofstream outf{ "MyFile.dat" };  
if (!outf)  
{  
    std::cerr << "File could not be opened";  
    return 1;  
}  
outf << "Hello, sweetie!" << '\n';  
return 0;
```

```
std::ifstream inf{ "MyFile.dat" };  
if (!inf)  
{  
    std::cerr << "File could not be opened\n";  
    return 1;  
}  
  
while (inf)  
{  
    std::string strInput;  
    std::getline(inf, strInput);  
}
```

std::flush

app

ate

binary

in

out

trunc

ФАЙЛОВЫЙ ВВОД/ВЫВОД

beg

cur

end

```
inf.seekg(14, ios::cur);  
inf.seekg(-18, ios::cur);
```

```
inf.seekg(0, ios::beg);  
inf.seekg(0, ios::end);
```

```
inf.tellg();
```

remove()

is_open()

